

DATABASE SCHEMA

& ERD DOCUMENTATION

E-Commerce Platform

Firebase Firestore

Generated April 29, 2026

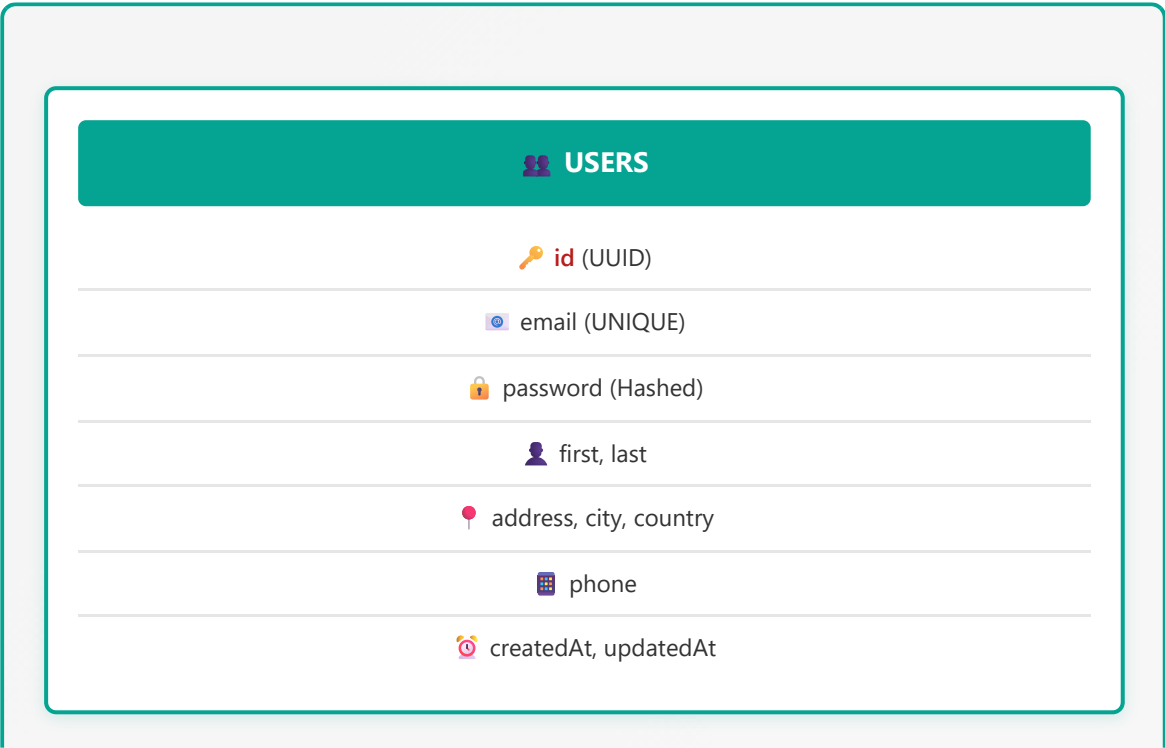
Project Overview & Architecture

This document describes the complete database schema and Entity Relationship Diagram (ERD) for the E-Commerce Platform. The system is built on Firebase Firestore, a NoSQL document database, providing scalable, real-time data synchronization.

Technology Stack

- **Database:** Firebase Firestore (NoSQL)
- **Storage:** Cloudinary (Image CDN)
- **Authentication:** Firebase Auth + Google OAuth2
- **Backend:** Node.js/Express
- **Password Security:** bcryptjs (10 salt rounds)

Entity Relationship Diagram



↓

One User : Many Orders

↓

PRODUCTS

🔑 id (UUID)

📄 name

🏷️ category

💰 price, oldPrice

🎨 color, brand

📦 inStock

🖼️ images[]

ORDERS

🔑 id (UUID)

👤 userId (FK)

📋 order_code

📦 items[]

💰 totalPrice

📊 status

🚚 deliveryAddress

3 Collections

Users, Products, Orders

1 Relationship

User → Orders (1:N)

Collections Detail: Users & Products

Users Collection

Manages user accounts, authentication, and profile information. Each user has a unique UUID and email address.

Field	Type	Constraints	Description
id	String	PK, UUID	Unique user identifier
email	String	UNIQUE, INDEXED	Email address (lowercase)
password	String	Hashed	bcryptjs hash (10 rounds)
first, last	String	Optional	Full name
phone	String	Optional	Contact number
address, city, country	String	Optional	Shipping address
createdAt, updatedAt	Timestamp	SERVER_GENERATED	Account timestamps

Products Collection

Maintains product catalog with pricing, inventory status, and images hosted on Cloudinary.

Field	Type	Constraints	Description
id	String	PK, UUID	Unique product ID
name	String	Required	Product title
category	String	INDEXED	phones, laptops, accessories, etc.
price, oldPrice	Number	Required	Current & original price
brand, color	String	Optional	Product attributes
inStock	Boolean	Default: true	Availability
images	Array<String>	Optional	Cloudinary URLs

Product Categories

phones • laptops • accessories • headphones • watches • kitchen

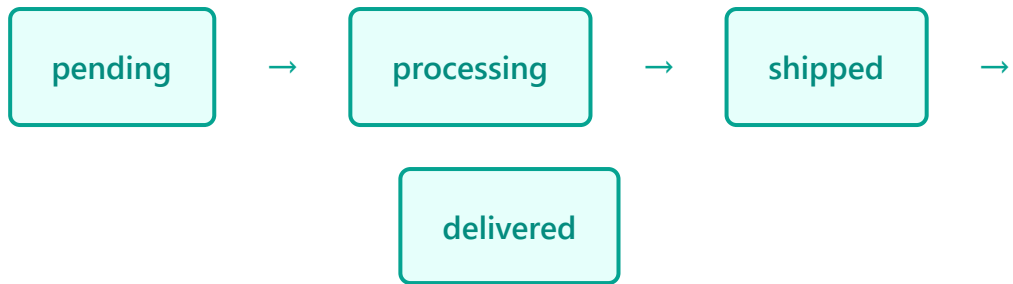
Orders Collection & Data Types

Orders Collection

Tracks customer orders with items, status workflow, and delivery information. Each order is linked to a user via `userId`.

Field	Type	Constraints	Description
<code>id</code>	String	PK, UUID	Order ID
<code>order_code</code>	String	UNIQUE	Human-readable code
<code>userId</code>	String	FK, INDEXED	Reference to Users
<code>items</code>	Array<Object>	Required	Order line items
<code>totalPrice</code>	Number	Required	Total amount
<code>status</code>	String	INDEXED, Enum	pending, processing, shipped, delivered
<code>paymentMethod</code>	String	Optional	Payment type
<code>createdAt,</code> <code>updatedAt</code>	Timestamp	SERVER_GENERATED	Order timestamps

Order Status Workflow



Data Types Reference

Type	Description
<code>String</code>	Text data (emails, names, descriptions)
<code>Number</code>	Integer or decimal values (prices, quantities)
<code>Boolean</code>	true/false values (inStock, flags)
<code>Timestamp</code>	Date-time values (Firebase <code>FieldValue.serverTimestamp()</code>)
<code>Array<String></code>	List of values (images, categories)
<code>Object</code>	Key-value pairs (deliveryAddress, order items)

Relationships & Implementation

Database Constraints

Primary Keys

- Users.id (UUID)
- Products.id (UUID)
- Orders.id (UUID)

Foreign Keys

- Orders.userId → Users.id

Unique Constraints

- Users.email
- Orders.order_code

Indexed Fields

- Users.email
- Products.category
- Orders.userId
- Orders.status

Key Features

- **Authentication:** Email/password or Google OAuth2. Passwords hashed with bcryptjs (10 salt rounds)
- **Image Storage:** Cloudinary CDN for product images, external storage reduces Firestore load
- **Real-time Sync:** Firebase Firestore provides automatic real-time updates
- **Server Timestamps:** All timestamps generated server-side to prevent client clock skew
- **Query Optimization:** Strategic indexing on userId, status, category for fast queries

- **Scalability:** Firestore automatically scales to millions of documents

Query Patterns

Get user orders: `Orders.where('userId', '==', userId)`

Find by category: `Products.where('category', '==', category)`

Filter by status: `Orders.where('status', '==', status)`

Data Integrity Rules

- Emails stored in lowercase for consistency
- Passwords never stored in plain text (bcryptjs hashing)
- Timestamps server-generated (Firestore `FieldValue.serverTimestamp()`)
- Order status follows defined workflow
- Product items maintain price snapshot at purchase time
- All IDs are UUIDs for global uniqueness